

D01_L02 Drive Observation Evaluation Guide

Lesson Objective

D01_L02 introduces a pure, stateless `DriveObservationEvaluator`. Its single responsibility is to determine whether a supplied immutable `DriveObservation` represents a stopped drivetrain using a tolerance supplied by the caller.

The lesson inherits directly from `D01_L01_Drive_Observation_Boundary`. The inherited observation

boundary remains frozen, and all 12 inherited production Java files remain byte-identical.

Architecture

The control path remains unchanged:

```
Driver
-> Xbox Controller
-> controls
-> commands
-> DriveSubsystem
-> DriveIO
-> Hardware or Simulation
```

The read-only evaluation path is:

```
DriveIO
-> DriveIOInputs
-> DriveSubsystem
-> DriveObservation
-> DriveObservationEvaluator
-> boolean
```

Dependency direction is one-way. The evaluator depends only on the immutable domain snapshot. It does not depend on `DriveSubsystem`, `DriveIO`, hardware, simulation, telemetry, or publishing. It cannot alter the drivetrain control path.

Responsibility and Contract

Type:

```
public final class DriveObservationEvaluator
```

Method:

```
public boolean isStopped(
    DriveObservation observation,
    double outputTolerance)
```

The evaluator:

- requires a non-null observation;
- requires a finite, non-negative tolerance;
- accepts both `0.0` and `-0.0` as zero tolerance;
- returns `false` if either observed output is non-finite;
- evaluates the left and right outputs independently;
- returns `true` only when both absolute outputs are less than or equal to the tolerance;
- uses inclusive boundary equality;
- does not average outputs, retain a snapshot, or own mutable state.

Input Validation

Input	Result
Null observation	<code>NullPointerException</code>
Negative tolerance	<code>IllegalArgumentException</code>
NaN tolerance	<code>IllegalArgumentException</code>
Positive or negative infinite tolerance	<code>IllegalArgumentException</code>
<code>0.0</code> or <code>-0.0</code> tolerance	Accepted
Non-finite left or right output	<code>false</code>

Inclusive Tolerance Behavior

Each side uses:

```
Math.abs(output) <= outputTolerance
```

Equality is intentionally accepted. With a tolerance of `0.1`, outputs of `0.1` and `-0.1` are both within tolerance. Either side exceeding the tolerance makes the complete evaluation `false`.

Exact Source Impact

Created:

- `src/main/java/frc/robot/observation/drive/DriveObservationEvaluator.java`

Modified production Java:

- None

Inherited domain snapshot:

- `src/main/java/frc/robot/observation/drive/DriveObservation.java`

Complete Source: DriveObservationEvaluator.java

```
// Copyright (c) FIRST and other WPILib contributors.
// Open Source Software; you can modify and/or share it under the terms of
// the WPILib BSD license file in the root directory of this project.

/**
 * Author: SSIS
 * Mentor: SSIS
 */

package frc.robot.observation.drive;

import java.util.Objects;

/**
 * Evaluates immutable drivetrain observations.
 */
public final class DriveObservationEvaluator {
    /**
     * Returns whether both drivetrain outputs are within the supplied stopping tolerance.
     *
     * @param observation immutable drivetrain observation
     * @param outputTolerance non-negative finite stopping tolerance
     * @return true when both finite outputs are within the inclusive tolerance
     */
    public boolean isStopped(
        DriveObservation observation,
        double outputTolerance) {
        Objects.requireNonNull(
            observation,
            "observation");

        if (!Double.isFinite(outputTolerance)
            || outputTolerance < 0.0) {
            throw new IllegalArgumentException(
                "outputTolerance must be finite and non-negative");
        }

        double leftOutput = observation.leftAppliedOutput();
        double rightOutput = observation.rightAppliedOutput();

        if (!Double.isFinite(leftOutput)
            || !Double.isFinite(rightOutput)) {
            return false;
        }

        return Math.abs(leftOutput) <= outputTolerance
            && Math.abs(rightOutput) <= outputTolerance;
    }
}
```

Complete Inherited Source: DriveObservation.java

```
// Copyright (c) FIRST and other WPILib contributors.
// Open Source Software; you can modify and/or share it under the terms of
// the WPILib BSD license file in the root directory of this project.

/**
 * Author: SSIS
 * Mentor: SSIS
 */

package frc.robot.observation.drive;

/**
 * Provides an immutable drivetrain observation from the latest completed subsystem periodic
 * update.
 *
 * @param leftAppliedOutput normalized, dimensionless left-side applied output
 * @param rightAppliedOutput normalized, dimensionless right-side applied output
 */
public record DriveObservation(
    double leftAppliedOutput,
    double rightAppliedOutput) {}
```

Build Verification

Commands:

```
.\gradlew.bat clean --no-daemon
.\gradlew.bat build --no-daemon
```

Result: **PASS** (BUILD SUCCESSFUL).

Simulation Verification

A temporary external harness exercised `DriveIOSim`, immutable observations, and the evaluator.

It completed 25 checks successfully, including:

- zero and signed-zero behavior;
- values inside and exactly at tolerance;
- left or right values outside tolerance;
- asymmetric, negative, and mixed outputs;
- non-finite outputs;
- null and invalid-tolerance exceptions;
- repeated evaluation without `updateInputs()` calls;
- unchanged simulated outputs after evaluation;
- immutable previously captured snapshots;
- zero declared evaluator fields.

The harness and every generated artifact were removed after verification.

Explicit Exclusions

D01_L02 does not introduce:

- drivetrain control or mutation;
- direct subsystem, IO, hardware, or simulation access from the evaluator;
- NetworkTables, SmartDashboard, Glass, telemetry, or publishing;
- logging;
- encoders, gyro, odometry, pose, or physics;
- timestamps, event buses, observer patterns, or generic evaluator abstractions;
- RobotContainer wiring or hidden tolerance constants.

Verification Status

- Architecture Review: PASS
- Implementation: PASS
- Production Build: PASS
- Simulation Verification: PASS
- Source Regression: PASS
- Documentation: PASS
- Driver Station / Glass: NOT TESTED
- Real Robot Verification: NOT TESTED
- Git Commit: NOT CREATED
- Git Push: NOT PERFORMED